

Poniżej przedstawiam kompletną, szczegółowo opisaną architekturę backendu systemu TipJar, z podziałem na poszczególne warstwy, moduły, integracje z usługami zewnętrznymi oraz rozwiązania wspierające skalowalność, bezpieczeństwo i utrzymanie systemu.

1. Ogólny Podział i Podejście Architektoniczne

Backend TipJar został zaprojektowany w oparciu o podejście ****mikrouslugowe**** (lub modularne, na mniejszą skalę – zależnie od planowanej ekspansji) z wyraźnym rozdzieleniem warstwy API, logiki biznesowej oraz integracji z zewnętrznymi serwisami. Główne moduły to:

- ****API Gateway / Entry Point**** – centralny punkt wejścia dla wszystkich żądań, odpowiedzialny za autentykację, routing, logowanie i kontrolę dostępu.
- ****Serwisy Kluczowe (Mikrouslugi)****, takie jak:
 - ****User Service**** – obsługa rejestracji, profili, zarządzania kontami i integracji z portfelami (Circle Wallet).
 - ****Payment Service**** – logika obsługi transakcji, mikropłatności, sponsorowania opłat (gas) oraz integracja z API Circle i potencjalnie CCTP.
 - ****Notification Service**** – wysyłka powiadomień w czasie rzeczywistym (np. WebSocket lub usługi push) o zdarzeniach płatności.
- ****Warstwa Integracyjna**** – moduł odpowiedzialny za komunikację z zewnętrznymi API:
 - ****Circle API**** – tworzenie i zarządzanie portfelami, inicjowanie transakcji.
 - ****Gas Station API oraz mechanizmy CCTP**** – dla optymalizacji opłat transakcyjnych.
- ****Infrastruktura Danych**** – baza danych PostgreSQL do przechowywania danych użytkowników, transakcji oraz Redis jako system cache'owania i kolejek (np. dla asynchronicznych zadań przetwarzania) oraz ewentualnego rate limiting.

2. Szczegółowy Opis Poszczególnych Komponentów

A. API Gateway / Entry Point

- **Technologia:**

- Realizowany za pomocą rozwiązań opartych na **Express** lub **NestJS** (w Node.js z TypeScriptem), co pozwala na silną typizację, lepszą organizację kodu i łatwiejsze skalowanie.

- **Główne zadania:**

- **Routing:** Kierowanie przychodzących żądań HTTP/REST (lub GraphQL) do odpowiednich serwisów.

- **Autoryzacja i uwierzytelnianie:** Implementacja kontroli dostępu przy pomocy JWT, middleware do walidacji tokenów oraz ewentualne wsparcie OAuth.

- **Logowanie i monitoring:** Logowanie zdarzeń oraz błędów, zbieranie metryk (np. przy wykorzystaniu Winston, Morgan, Prometheus).

- **Ochrona:** Rate limiting, obsługa CORS, walidacja danych wejściowych.

B. Serwisy Kluczowe (Mikrouslugi)

1. User Service

- **Funkcjonalności:**

- **Rejestracja i logowanie:** Umożliwia tworzenie kont, weryfikację danych, generowanie JWT.

- **Zarządzanie profilami:** Aktualizacja danych profilu, konfiguracja konta, integracja z Circle Wallet (tworzenie/łączenie portfela w trakcie rejestracji).

- **Bezpieczeństwo i sesje:** Wykorzystanie Redis do przechowywania sesji lub krótkoterminowych danych uwierzytelniających.

- **Architektura:**

- **Kontrolery (Controllers):** Odpowiedzialne za przyjmowanie żądań (np. POST /users, GET /users/me).

- **Serwisy (Services):** Logika biznesowa, np. walidacja danych, łączenie z zewnętrznymi API (Circle) przez dedykowany moduł integracyjny.

- **Repozytoria:** Klasa lub warstwa ORM (np. TypeORM lub Prisma) realizująca operacje na bazie PostgreSQL.

2. Payment Service

- **Funkcjonalności:**

- **Obsługa transakcji:** Inicjacja, monitorowanie i rejestrowanie micropłatności (napiwków).

- **Integracja z Circle API:** Moduł do komunikacji z zewnętrznym API służącym do zarządzania portfelami i transakcjami, w tym podpisywanie i monitorowanie transakcji.

- **Sponsorowanie opłat:** Mechanizm wywołujący operacje na Gas Station API, który pozwala fanom wysyłać napiwki bez martwienia się o koszty „gas”.

- **Obsługa konwersji (CCTP):** Automatyczne przekonwertowanie USDC między sieciami (np. z Ethereum na Avalanche), jeśli twórca preferuje odbiór na innym łańcuchu.

- **Architektura:**

- **Endpointy API:** RESTful endpoints np. POST /payments, GET /payments/status.

- **Asynchroniczny processing:** W przypadku intensywnego ruchu warto zastosować kolejki (Redis jako message broker, RabbitMQ) do przetwarzania płatności i webhooków z Circle.

- **Moduł integracyjny:** Abstrakcja komunikacji z Circle oraz Gas Station API – odpowiedzialna za autoryzację żądań, obsługę webhooks i retries w przypadku błędów komunikacyjnych.

- **Rejestr transakcji:** Zapisywanie szczegółowych logów transakcji w bazie danych (PostgreSQL) wraz z metadanymi (czas, status, kwota, identyfikatory użytkowników).

3. Notification Service

- **Funkcjonalności:**

- **Real-time Notifications:** Mechanizm powiadomień na żywo o zdarzeniach, np. nowo otrzymanych napiwkach.

- **Wsparcie dla różnych protokołów:** Wdrożenie WebSocket (np. przy użyciu Socket.IO) lub integracja z zewnętrznymi usługami push (Pusher, Firebase Cloud Messaging).

- **Obsługa kolejki zdarzeń:** Po otrzymaniu potwierdzenia transakcji przez Payment Service, odpowiedni event jest wysyłany do Notification Service, który rozpoczyna transmisję powiadomień do sesji użytkownika.

- **Architektura:**

- **Serwer WebSocket:** Umożliwia stałe połączenie z klientami.

- **Integracja z API Gateway:** Kiedy żądanie płatności zakończy się sukcesem, Notification Service jest wywoływana w sposób asynchroniczny (np. poprzez event bus lub message queues).

C. Warstwa Integracyjna (Third-Party API)

- **Circle API Integration:**

- **Moduł klienta API:** Implementacja przy użyciu popularnych bibliotek (np. Axios) konfigurujących bazowy URL oraz nagłówki dla autentykacji.

- **Zarządzanie kluczami API:** Bezpieczne przechowywanie w zmiennych środowiskowych i ewentualnie w dedykowanym managerze sekretów.

- **Obsługa webhooków:** Endpoints rejestrujące powiadomienia z Circle – np. potwierdzenia transakcji, zmiany statusu portfela.

- **Gas Station / CCTP Integration:**

- **Moduł sponsorowania gas:** Okresowe sprawdzanie cen gazu na niskokosztowych sieciach (Polygon, Solana) i inicjowanie sponsorowania opłat w mikropłatnościach.

- **Automatyczna konwersja:** Mechanizm wykrywający potrzebę przekonwertowania USDC między łańcuchami w trybie quasi-real-time, z odpowiednimi fallbackami w przypadku niedostępności usługi.

D. Infrastruktura Danych

- **Baza Danych – PostgreSQL:**

- **Schematy:**

- **Users:** Informacje o użytkownikach, identyfikatory, dane profilu, informacje o portfelach (adresy, statusy weryfikacji).

- **Payments:** Log transakcji, dane transakcji, metadane związane z opłatami gas, statusy.

- **Configurations:** Dane konfiguracyjne systemu, klucze API, ustawienia integracyjne.

- **Połączenie z ORM:** Użycie TypeORM lub Prisma zapewnia wygodną abstrakcję bazy i silne typowanie.

- **Cache i Kolejki – Redis:**

- **Caching:** Przechowywanie sesji, tokenów, wyników zapytań często używanych (np. licznik płatności).

- **Message Queues:** Kolejowanie zdarzeń (np. synchronizacja statusów płatności, wysyłka powiadomień) dla zapewnienia asynchronicznego przetwarzania.

E. Warstwa Middleware, Loggerów i Monitoringu

- **Middleware:**

- Obsługa błędów (globalny handler błędów).
- Walidacja danych wejściowych (np. przy użyciu bibliotek typu Joi lub class-validator w NestJS).
- Autoryzacja żądań – sprawdzanie tokenów JWT oraz uprawnień.

- **Logowanie:**

- Implementacja logowania (Winston, Morgan) z możliwością przesyłania logów do centralnych systemów (np. ELK Stack, Sentry) dla wykrywania i analizy problemów.

- **Monitoring i metryki:**

- Integracja z Prometheus i Grafana do monitorowania zużycia zasobów, opóźnień odpowiedzi, wycieków pamięci, etc.
- Wdrożenie zasad alarmowych i alertów w wypadku przekroczenia określonych progów (np. liczby błędów 5xx, spadek wydajności).

F. Skalowalność, Bezpieczeństwo i CI/CD

- **Skalowalność:**

- **Containerization:** Cały backend jest konteneryzowany przy użyciu Docker, co umożliwia łatwe skalowanie przy użyciu Kubernetes lub Docker Compose na mniejszą skalę.
- **Load Balancery:** Przed API Gateway można zastosować load balancer (np. Nginx, HAProxy) dla rozdzielenia ruchu między instancje.
- **Horizontal Scaling:** Usługi stateless (np. API Gateway, Payment Service) można łatwo skalować poziomo.

- **Bezpieczeństwo:**

- ****Przechowywanie sekretów:**** Wszystkie klucze, tokeny dostępowe oraz klucze API są przechowywane w bezpiecznych środowiskach (np. HashiCorp Vault, AWS Secrets Manager).

- ****Bezpieczne połączenia:**** Wymuszenie HTTPS dla wszystkich komunikacji między klientami a backendem.

- ****Ochrona przed atakami:**** Wdrożenie mechanizmów rate limiting, filtrowania adresów IP, monitoringu podejrzanej aktywności.

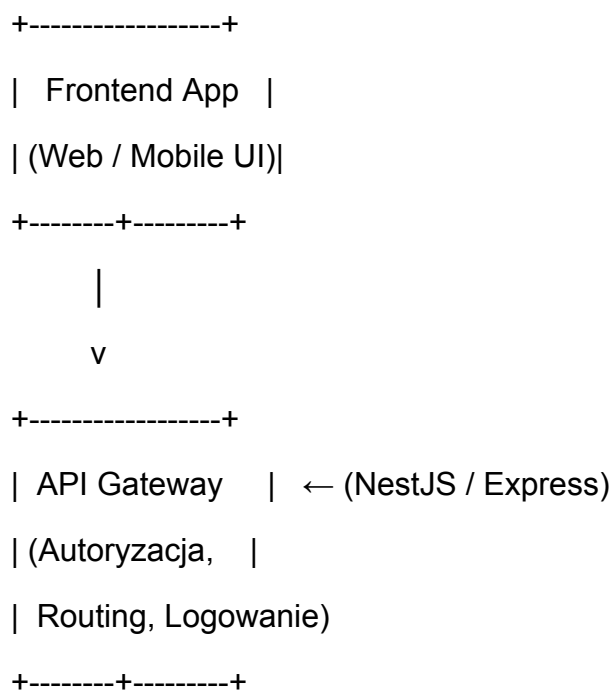
- ****Proces CI/CD:****

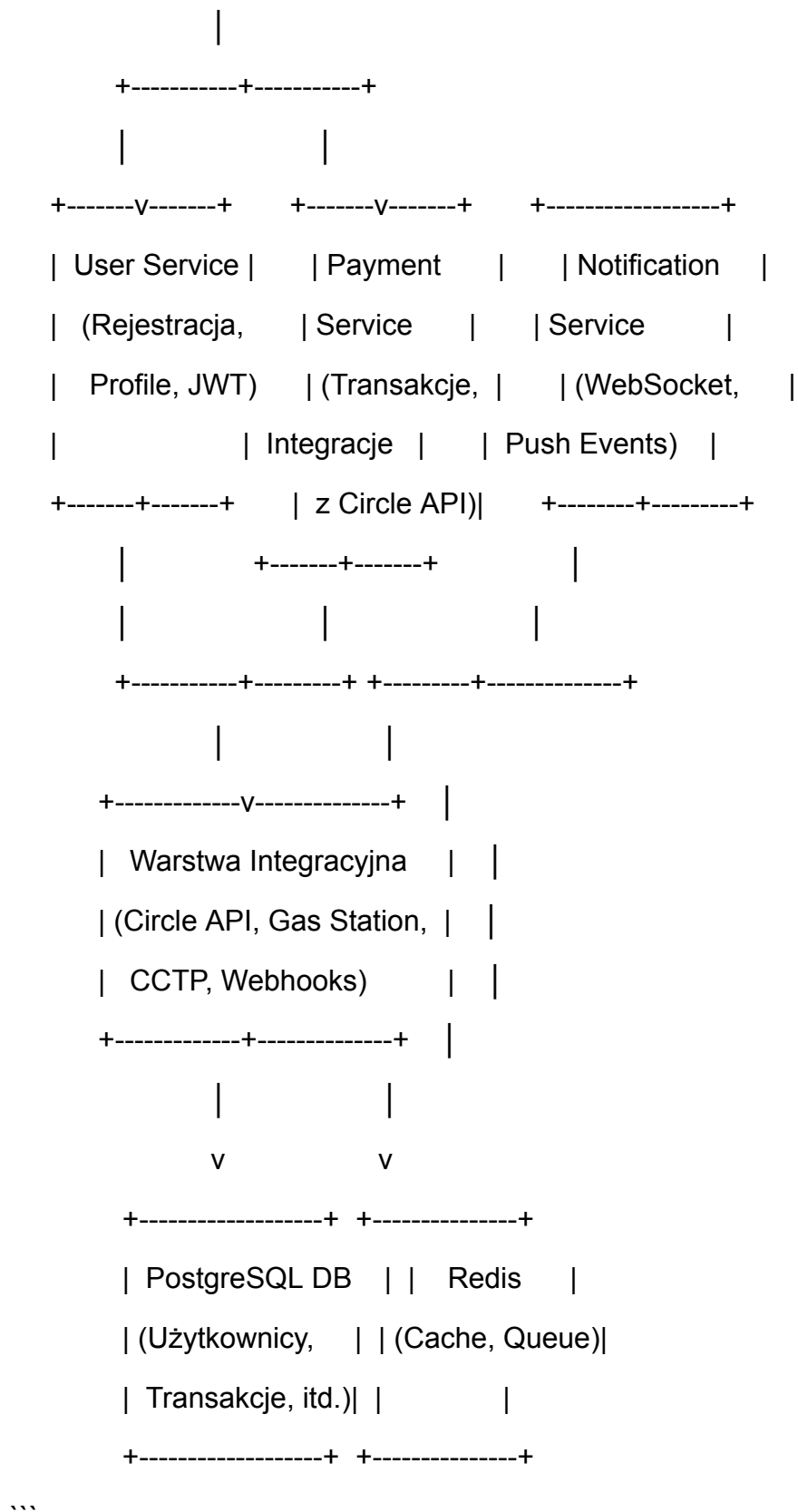
- ****Budowanie i testowanie:**** Pipeline skonfigurowany w GitHub Actions lub innym CI/CD (np. Jenkins, GitLab CI) uruchamia testy jednostkowe, integracyjne oraz budowanie obrazów Docker.

- ****Deploy:**** Automatyczne wdrażanie na środowiskach staging i produkcyjnym, z możliwością rollbacku w przypadku wykrycia błędów.

3. Przykładowy Diagram Architektury

```plaintext







## ## 4. Przebieg Przetwarzania Żądania

### 1. \*\*Wejście w API Gateway:\*\*

Użytkownik (twórca lub fan) wysyła żądanie do API Gateway, gdzie następuje weryfikacja tokenów JWT, walidacja parametrów, logowanie żądania oraz przekierowanie do odpowiedniego serwisu.

### 2. \*\*Interakcja z User Service:\*\*

Na etapie rejestracji lub logowania, User Service przy pomocy ORM zapisuje dane w PostgreSQL, generując jednocześnie token dostępu, a integrowany moduł wywołuje Circle API w celu utworzenia powiązanego portfela (przy użyciu bezpiecznego klienta HTTP).

### 3. \*\*Operacja Płatnicza:\*\*

Gdy fan wysyła napiwek, żądanie trafia do Payment Service, gdzie:

- Inicjujemy rejestrację transakcji w bazie danych.
- Moduł integracyjny wywołuje Circle API, sponsoruje opłatę gas przez Gas Station API i w razie potrzeby przetwarza operację konwersji (CCTP).
- W odpowiedzi system zapisuje status transakcji oraz wyzwala event dla Notification Service.

### 4. \*\*Powiadomienia:\*\*

Notification Service, pobierając event z kolejki Redis (lub przy pomocy dedykowanego event busa), wysyła do klienta informację o nowym napiwku, co jest wyświetlane na dashboardzie twórcy lub jako powiadomienie w aplikacji.

---

## ## 5. Deployment i Utrzymanie

- **\*\*Konteneryzacja i Orkiestracja:\*\***

Wszystkie komponenty są spakowane w obrazy Docker. Na początkowym etapie można używać Docker Compose, a przy rozwoju Magento Kubernetes lub Docker Swarm umożliwią horizontal scaling.

- **\*\*CI/CD:\*\***

Pipeline budujący obraz, uruchamiający testy i deployujący nową wersję usługi do środowisk staging/production. Konfiguracja GitHub Actions (lub innego narzędzia) umożliwia szybkie iteracje i automatyczne roll-backi.

- **\*\*Monitoring i Logowanie:\*\***

Implementacja centralnego logowania (ELK Stack, Sentry) oraz systemu metryk (Prometheus, Grafana) zapewnia monitorowanie działania systemu i szybkie reagowanie na problemy.

---

## ## Podsumowanie

Kompleksowa architektura backendu TipJar opiera się na modularnym podejściu, w którym każdy komponent (User, Payment, Notification) jest wydzieloną mikrousługą, ułatwiającą skalowanie, wdrażanie nowych funkcji oraz utrzymanie bezpieczeństwa. Integracja z zewnętrznymi API (Circle, Gas Station, ewentualnie systemy do konwersji tokenów) odbywa się poprzez dedykowane moduły, a całość wspiera dobrze zaprojektowana warstwa monitoringu, logowania oraz CI/CD. Taki model umożliwia nie tylko płynne przetwarzanie mikropłatności, ale także szybką adaptację systemu do rosnącej liczby użytkowników i dynamicznych zmian na rynku technologii web3.

Czy warto zagłębić się w szczegóły implementacyjne jednego z modułów (np. przykładowy moduł integracji z Circle API lub konfigurację mechanizmu kolejkowania zdarzeń przy użyciu Redis)?